

Full detailed structured draft, from scratch to final workflow

Project title

Urban Matrix Parametric Modular Housing Generator

Author

Prasanna Chaurasia
Urban Matrix

Project summary

This project is a browser-based parametric architectural design generator built in **p5.js**. It creates a modular housing concept from a simple box module and expands it into a full housing block with site boundary control, modular massing, façades, green roofs, public realm, environmental overlays, performance logic, material systems, data exports and architectural drawing exports.

The project is structured as a design workflow rather than a single static model. It starts from a site boundary, defines the buildable envelope, generates modular housing inside the plot, adds architectural details, tests passive environmental strategies, evaluates conceptual performance indicators, and exports documentation drawings such as top plan, site plan, masterplan, elevations, sections, moodboard and technical sheets.

Why this project is important

The main architectural value of this project is that it transforms modular housing from a fixed object into an **iterative design system**. Instead of manually modelling one option, the designer can generate multiple controlled options, compare massing, density, setbacks, courtyards, green roofs, public space, façade response, energy targets, carbon logic and material systems.

This aligns strongly with contemporary architectural workflows where design is increasingly shaped through **parametric modelling, BIM thinking, environmental analysis, performance comparison and structured information exchange**. ISO 19650 explains BIM-related information management as a framework for exchanging, recording, versioning and organising information across actors, which supports the logic of exporting structured JSON, CSV and module data from this generator.

Stage-by-stage process

Stage 00, Site Boundary and Planning Envelope

The project begins by defining the site. This is essential because a building cannot be designed responsibly without understanding its plot, boundaries, access, setbacks, roads, landscape and public realm.

The script includes:

- Site width
- Site depth
- Front setback
- Rear setback
- Left setback
- Right setback
- Road width
- Footpath width
- Landscape buffer
- Public space ratio
- Metric grid spacing
- Auto-fit building toggle
- Site boundary warning
- North arrow
- Scale bar
- Buildable envelope

This solves the earlier issue where increasing rows and columns caused the building to overlap with roads or urban context. The model now checks the buildable envelope and automatically limits rows and columns when **Auto-fit Site** is active.

Stage 01, Modular Massing

The massing stage creates the housing from repeated modules. Each module has width, depth, height and spacing. The generator then stacks modules into columns, rows and floors.

The script includes:

- Module width in metres
- Module depth in metres
- Floor height in metres
- Module gap in metres
- Number of columns
- Number of rows
- Number of floors
- Density
- Floor setback
- Irregular offset
- Terrace voids
- Courtyard size
- Model scale

The logic is:

site boundary → buildable envelope → safe grid → modular units → stacked massing

This makes the design more architectural because the modular block is no longer floating randomly. It is constrained by a real site planning system.

Stage 02, Architectural Detailing

This stage adds architectural identity. The earlier version had a problem where the front façade had detail but the rear and side elevations looked blank. This final script corrects that by adding **four-sided façade logic**.

The script now generates details on:

- Front façade
- Rear façade
- Left façade
- Right façade
- Courtyard-facing sides

It includes:

- Windows
- Glazing
- Frames
- Balconies
- Railings
- Jaali panels
- Vertical fins
- Horizontal louvres
- Parapets
- Roof slabs
- Core and entrance marker
- Shared decks

This makes the design more believable from all sides and improves export quality for front, rear, left and right elevations.

Stage 03, Environmental Logic

This stage adds environmental diagrams and passive design logic.

It includes:

- Sun direction
- Sun altitude
- Shadow overlay
- Passive façade response
- Ventilation arrows
- Rainwater arrows
- Green roof percentage
- Planters
- Vegetation scale
- Tree count
- Public space scale
- Analysis overlay

The earlier black film in the model was the shadow analysis overlay. In this final logic, it is controlled and labelled as an environmental diagram rather than appearing as a random black sheet.

The sun diagram has also been moved far away from the housing block, so it does not overlap the building.

Stage 04, Codes, Materials and Performance

This stage is a conceptual performance layer. It does not claim legal compliance, but it organises design decisions around:

- Code strictness
- Accessibility logic
- Fire safety awareness
- Energy target
- Carbon target
- PV roof area
- Water reuse
- Urban greening
- Comfort
- Material system
- Iteration comparison

The script includes four material systems:

- Low Carbon Timber Hybrid
- Precast Concrete Modular
- Steel Modular Frame
- Mixed Local Materials

Each system has a conceptual carbon factor and material description. WorldGBC's whole-life carbon framework is relevant here because it defines the importance of considering both operational and embodied carbon across the building lifecycle.

The important professional wording is:

This generator is informed by building-code logic, accessibility logic, passive design and carbon principles, but it is not a certified compliance tool.

Stage 05, Data Export System

This stage turns the visual model into structured design data.

The script exports:

- JSON project data
- OBJ-like conceptual geometry
- CSV module schedule
- 50-option iteration log
- Current PNG screenshot
- Performance data

Material information
Region information
Module positions
Module dimensions

This is where the project becomes more aligned with computational design and BIM thinking. It does not become a full BIM model automatically, but it creates structured data that can later support Rhino, Grasshopper, Blender, Revit or IFC workflows.

Stage 06, Architectural Drawing and Documentation Export

This is the architectural documentation stage.

The script exports:

Top plan
Site plan
Masterplan
Front elevation
Rear elevation
Left elevation
Right elevation
Section A-A
Section B-B
Moodboard
Technical sheet
All sheets together

The export system also includes:

High-resolution export multiplier
Line weight
Text size
Dimension visibility
Label visibility
Scale bar
North arrow
Sheet footer
Urban Matrix branding

This moves the project towards RIBA Stage 2 and partly Stage 3. RIBA describes its Plan of Work as organising briefing, designing, constructing and operating building projects into eight stages with stage outcomes, core tasks and information exchanges.

RIBA stage alignment

This project mainly supports:

RIBA Stage 0, Strategic Definition

The tool helps test whether modular housing is a suitable strategy for a site.

RIBA Stage 1, Preparation and Briefing

The tool helps define site constraints, region, public realm, performance ambition and design targets.

RIBA Stage 2, Concept Design

The tool generates massing, façade logic, environmental response, modular arrangement, public space and material direction.

RIBA Stage 3, Spatial Coordination, partly

The drawing exports, sections, elevations, site plan and module schedules can support early spatial coordination. However, true Stage 3 would still require properly coordinated architectural, structural, MEP, fire and planning information.

It does **not** yet represent:

RIBA Stage 4 technical design

RIBA Stage 5 manufacturing and construction

RIBA Stage 6 handover

RIBA Stage 7 use

Those later stages need professional technical design, specification, coordination, statutory compliance, construction details and consultant input.

Global code and authenticity statement

This project is globally adaptable. It is not UK-only. The region dropdown allows the model to respond conceptually to different climatic and planning contexts, such as Kuttanad, London, Manchester, Jaipur or a generic global site.

However, the script must not claim that the design is automatically compliant with any country's building regulations. The International Code Council describes the I-Codes as coordinated model codes used widely, but real adoption and enforcement depend on local jurisdictions. This supports the project's wording that all outputs must be checked against the selected country, city and local authority.

Correct professional wording:

This project is an authentic conceptual and computational design study. It is globally adaptable and region-responsive, but final compliance must always be verified against the local planning authority, building code, fire code, accessibility code, energy code, structural code, environmental assessment method and professional consultant advice.

Reflection, Prasanna Chaurasia and Urban Matrix alignment

This project aligns strongly with **Prasanna Chaurasia** and **Urban Matrix** because it combines architecture, BIM thinking, AI-assisted workflows, computational design, sustainability, urban design and visual communication.

It reflects your current direction in several ways:

It shows architectural design thinking through modular housing, terraces, public spaces and façades.

It shows computational capability through sliders, parameters, controlled iterations and exportable data.

It shows BIM awareness through structured module schedules, JSON data, CSV export and information logic.

It shows sustainability awareness through green roofs, PV logic, water reuse, urban greening, carbon scoring and passive design.

It shows urban design thinking through site boundary, public plaza, road edge, pocket park, pedestrian paths and masterplan export.

It shows portfolio value because it produces visual outputs, diagrams, drawings, schedules and conceptual performance data.

For Urban Matrix, this becomes more than a design model. It becomes a **digital design system** that can demonstrate how AI, parametric modelling, BIM thinking and sustainability can support early-stage architectural decision-making.

A strong reflection statement would be:

As Prasanna Chaurasia, Co-founder of Urban Matrix, this project represents my interest in combining architectural design, BIM workflows, computational thinking and sustainability-led urban design. The generator transforms a single modular housing unit into an adaptive residential system that can be tested across site conditions, climate logic, public realm priorities, material choices and conceptual performance criteria. It reflects my aim to use digital tools not only for visualisation, but for structured design thinking, iteration, documentation and future BIM-ready workflows.

Source and justification notes

The script uses p5.js because p5.js supports creative coding, [WEBGL](#) for 3D graphics, DOM controls such as sliders and dropdowns, and export-oriented creative workflows.

The design process is aligned with early RIBA stages because RIBA Plan of Work organises briefing, designing, constructing and operating building projects into eight structured stages. The data export logic is connected to BIM information management thinking because ISO 19650 describes concepts and principles for managing information using BIM, including exchanging, recording, versioning and organising information. The sustainability and carbon logic is conceptually aligned with whole-life carbon thinking because WorldGBC's Whole Life Carbon Vision sets out principles for total sector decarbonisation and net zero operational and whole-life carbon buildings.

Final p5.js Script

/*

=====

URBAN MATRIX PARAMETRIC MODULAR HOUSING GENERATOR V2.1

Prasanna Chaurasia | Urban Matrix

Paste this entire file into p5.js Web Editor as sketch.js.

Major corrections in V2.1:

1. Full canvas remains visible. Controls are now collapsible and stage-tabbed.

2. H hides/shows controls. C toggles clean screenshot mode.

3. Drawing dropdown now changes live camera/view direction.

4. No WEBGL text is drawn, avoiding the p5 WEBGL font error.

5. Infinite-loop risk is reduced by guarding dynamic loop steps.

6. Stage 00 site boundary includes Side A/B/C/D labels in exports.

7. 2D documentation exports include title blocks, dimensions,

floor levels, scale bars, legends and SVG vector export.

8. PV panels, context blocks and lift/stair core are labelled in drawings

and controlled by toggles.

9. Metric scale is explicit: 1 p5 unit = 0.1 metre.

Important professional note:

This is an early-stage conceptual parametric design and documentation tool.

It is not a certified planning, fire, structure, MEP, energy, BIM, statutory

approval, construction, cost or code-compliance tool. Final work must be

verified against local codes, consultants and authority requirements.

=====

*/

```
/* =====
```

GLOBAL PROJECT SETTINGS

```
===== */
```

```
const PROJECT_NAME = "Parametric Modular Housing Generator";
```

```
const BRAND_NAME = "Urban Matrix";
```

```
const AUTHOR_NAME = "Prasanna Chaurasia";
```

```
const LINKEDIN_URL = "PASTE_YOUR_LINKEDIN_URL_HERE";
```

```
const P5_TO_METRE = 0.1;
```

```
let controls = {};
```

```
let labels = {};
```

```
let toggles = {};
```

```
let selects = {};
```

```
let buttons = {};
```

```
let stagePanels = {};
```

```
let cnv, uiRoot, uiDrawer, uiTabs, hudPanel;
```

```
let uiVisible = true;
```

```
let drawerOpen = true;
```

```
let activeStage = "Stage 00";
```

```
let cleanMode = false;
```

```
let seedValue = 1807;
```

```
let iterationIndex = 1;
```

```
let maxIterations = 50;
```

```
let savedIterationLog = [];
```

```
let generatedModules = [];
```

```
let generatedMetrics = {};
```

```
let boundaryWarning = false;
```

```
let spacingWarning = false;
```

```
let groundY = 0;
```

```
let camYaw = -0.72;
```

```
let camPitch = -0.48;
```

```
let camDist = 1080;
```

```
let camTarget = { x: 90, y: -150, z: 20 };
```

```
let dragging = false;
```

```
let lastMouseX = 0;
```

```
let lastMouseY = 0;
```

```
const palette = {
```

```
  sky: [232, 232, 226],
```

```
  skyNight: [32, 38, 48],
```

```
  ground: [222, 214, 199],
```

```
  road: [70, 72, 72],
```

```
  footpath: [230, 216, 194],
```

```
  boundary: [218, 68, 54],
```

```
  envelope: [70, 148, 98],
```

```
  grass: [126, 171, 122],
```

```
  shrub: [86, 134, 82],
```

```
  trunk: [132, 98, 70],
```

```
  concrete: [214, 203, 188],
```

```
  clay: [203, 154, 115],
```

```
  brick: [188, 126, 98],
```

timber: [174, 124, 84],

stone: [193, 176, 154],

glass: [124, 154, 164],

rail: [74, 80, 82],

pv: [55, 82, 105],

water: [95, 150, 178],

shadow: [110, 116, 118]

};

const materialProfiles = {

"Low Carbon Timber Hybrid": {

structure: "CLT / glulam frame with recycled steel connectors",

facade: "Timber rainscreen, shaded balconies and low-carbon panels",

carbonFactor: 0.55,

colour: [184, 160, 116],

note: "Lower embodied carbon direction. Requires fire, acoustic and moisture verification."

},

"Precast Concrete Modular": {

structure: "Precast concrete volumetric or panelised modules",

facade: "Precast concrete, brick slips and insulated panels",

carbonFactor: 0.82,

colour: [205, 200, 190],

note: "Durable and robust. Embodied carbon must be reduced with efficient design."

},

"Steel Modular Frame": {

structure: "Light-gauge or hot-rolled steel modular frame",

facade: "Lightweight rainscreen, insulated cassettes and shading screens",

carbonFactor: 0.74,

```
    colour: [176, 188, 196],

    note: "Fast modular assembly. Thermal bridging and fire protection require verification."
  },

  "Mixed Local Materials": {

    structure: "Hybrid frame using locally available materials",

    facade: "Local brick, timber, clay tile, lime plaster or fibre-cement panels",

    carbonFactor: 0.68,

    colour: [203, 154, 115],

    note: "Regionally adaptable. Depends on verified local supply chains."
  }
};
```

```
const regionProfiles = {

  "Kuttanad, India | Capital: New Delhi": {

    shortName: "Kuttanad",

    climate: "humid tropical, flood-prone lowland",

    sunBias: 0.82,

    rainBias: 0.95,

    ventilationBias: 0.92,

    shadingBias: 0.88,

    pvBias: 0.72,

    waterBias: 0.96,

    plinthBias: 0.92,

    recommendedMaterial: "Mixed Local Materials",

    note: "Raised plinth, verandas, rainwater logic, shaded terraces and cross ventilation."
  },

  "London, United Kingdom | Capital: London": {

    shortName: "London",
```

climate: "temperate urban",

sunBias: 0.55,

rainBias: 0.65,

ventilationBias: 0.62,

shadingBias: 0.55,

pvBias: 0.52,

waterBias: 0.70,

plinthBias: 0.35,

recommendedMaterial: "Low Carbon Timber Hybrid",

note: "Urban greening, daylight balance, compact massing and public realm quality."

},

"Manchester, United Kingdom | Capital: London": {

shortName: "Manchester",

climate: "cool temperate, wet urban",

sunBias: 0.48,

rainBias: 0.78,

ventilationBias: 0.58,

shadingBias: 0.46,

pvBias: 0.48,

waterBias: 0.80,

plinthBias: 0.42,

recommendedMaterial: "Low Carbon Timber Hybrid",

note: "Rain resilience, sheltered shared space and durable envelope strategy."

},

"Jaipur, India | Capital: New Delhi": {

shortName: "Jaipur",

climate: "hot dry",

sunBias: 0.96,

```
rainBias: 0.30,

ventilationBias: 0.74,

shadingBias: 0.98,

pvBias: 0.92,

waterBias: 0.55,

plinthBias: 0.28,

recommendedMaterial: "Mixed Local Materials",

note: "Deep shade, jaali screens, courtyards, thermal mass and solar energy."

},

"Generic Global Site | Capital: Not Applicable": {

  shortName: "Generic Global Site",

  climate: "balanced generic",

  sunBias: 0.65,

  rainBias: 0.60,

  ventilationBias: 0.65,

  shadingBias: 0.65,

  pvBias: 0.60,

  waterBias: 0.60,

  plinthBias: 0.40,

  recommendedMaterial: "Steel Modular Frame",

  note: "Balanced modular design, flexible public realm and moderate greening."

}

};
```

/* =====

P5 SETUP AND DRAW

===== */

```
function setup() {  
  
  cnv = createCanvas(windowWidth, windowHeight, WEBGL);  
  
  cnv.elt.oncontextmenu = () => false;  
  
  cnv.elt.addEventListener("wheel", e => e.preventDefault(), { passive: false });  
  
  
  pixelDensity(2);  
  
  frameRate(30);  
  
  
  
  createUI();  
  
  applyRegionPreset();  
  
  applyOptimisationPreset();  
  
  resetCamera();  
  
}  
  
  
function draw() {  
  
  const view = selects.drawingType.value();  
  
  const viewMode = selects.viewMode.value();  
  
  
  if (viewMode === "Night") background(...palette.skyNight);  
  
  else background(...palette.sky);  
  
  
  
  updateCameraForDrawingView(view);  
  
  scale(value("zoom"));  
  
  setupLights();  
  
  
  
  generatedModules = [];  
  
  boundaryWarning = false;  
  
  spacingWarning = false;
```

```
drawSiteSystem3D();
```

```
if (checked("publicRealm")) drawPublicRealm3D();
```

```
if (checked("stage01")) drawModularHousing3D();
```

```
if (checked("context")) drawContextBuildings3D();
```

```
if (checked("stage03")) drawEnvironmental3D();
```

```
if (checked("stage04")) drawPerformanceOverlays3D();
```

```
updateMetrics();
```

```
updateHUD();
```

```
updateLabels();
```

```
}
```

```
/* =====
```

```
UI, COLLAPSIBLE DRAWER AND STAGE TABS
```

```
===== */
```

```
function createUI() {
```

```
  uiRoot = createDiv();
```

```
  uiRoot.id("urban-matrix-ui-root");
```

```
  uiRoot.style("position", "fixed");
```

```
  uiRoot.style("left", "0");
```

```
  uiRoot.style("right", "0");
```

```
  uiRoot.style("bottom", "0");
```

```
  uiRoot.style("z-index", "20");
```

```
  uiRoot.style("font-family", "Inter, Arial, sans-serif");
```

```
  uiRoot.style("pointer-events", "none");
```

```
const toolbar = createDiv();

toolbar.parent(uiRoot);

toolbar.style("pointer-events", "auto");

toolbar.style("display", "flex");

toolbar.style("align-items", "center");

toolbar.style("gap", "8px");

toolbar.style("padding", "8px 12px");

toolbar.style("background", "rgba(12,12,12,0.78)");

toolbar.style("border-top", "1px solid #9ddc7f");

toolbar.style("backdrop-filter", "blur(8px)");

toolbar.style("flex-wrap", "wrap");
```

```
buttons.drawer = makeButton(toolbar, "Hide Controls | H", toggleUIVisibility);

buttons.collapse = makeButton(toolbar, "Collapse Drawer", toggleDrawer);

buttons.clean = makeButton(toolbar, "Clean Mode | C", toggleCleanMode);

buttons.randomise = makeButton(toolbar, "Randomise 50 | R", randomiseMassing);

buttons.reset = makeButton(toolbar, "Reset Camera", resetCamera);

buttons.png = makeButton(toolbar, "High-res PNG | P", saveCurrentViewPNG);

buttons.json = makeButton(toolbar, "JSON | S", exportModuleJSON);

buttons.csv = makeButton(toolbar, "CSV", exportCSVSchedules);

buttons.obj = makeButton(toolbar, "OBJ-like | O", exportOBJLike);

buttons.sheetPNG = makeButton(toolbar, "Export Drawing PNG", exportSelectedDrawingPNG);

buttons.sheetSVG = makeButton(toolbar, "Export Drawing SVG", exportSelectedDrawingSVG);

buttons.allPNG = makeButton(toolbar, "All Sheets PNG", exportAllDrawingPNGs);

buttons.allSVG = makeButton(toolbar, "All Sheets SVG", exportAllDrawingSVGs);
```

```
addToolbarSelect(toolbar, "View", "viewMode", ["Day", "Night", "Architectural Diagram", "Carbon Diagram", "Performance Diagram", "Material Diagram"], "Day");
```

```
addToolbarSelect(toolbar, "Drawing", "drawingType", ["Current 3D", "Top Plan", "Ground Floor Plan", "Roof Plan",
"Site Plan", "Masterplan", "Front Elevation", "Rear Elevation", "Left Elevation", "Right Elevation", "Section A-A",
"Section B-B", "Moodboard", "Technical Sheet"], "Current 3D");

addToolbarSelect(toolbar, "Region", "region", Object.keys(regionProfiles), "Kuttanad, India | Capital: New Delhi",
applyRegionPreset);

addToolbarSelect(toolbar, "Optimisation", "optimisationMode", ["Balanced", "Maximum Performance", "Low Carbon",
"Comfort Priority", "Urban Greening"], "Balanced", applyOptimisationPreset);

addToolbarSelect(toolbar, "Material", "materialSystem", Object.keys(materialProfiles), "Mixed Local Materials");


uiDrawer = createDiv();

uiDrawer.parent(uiRoot);

uiDrawer.style("pointer-events", "auto");

uiDrawer.style("background", "rgba(12,12,12,0.72)");

uiDrawer.style("backdrop-filter", "blur(10px)");

uiDrawer.style("color", "white");

uiDrawer.style("max-height", "42vh");

uiDrawer.style("overflow-y", "auto");

uiDrawer.style("padding", "10px 12px 14px 12px");


uiTabs = createDiv();

uiTabs.parent(uiDrawer);

uiTabs.style("display", "flex");

uiTabs.style("gap", "6px");

uiTabs.style("flex-wrap", "wrap");

uiTabs.style("margin-bottom", "10px");


const stages = ["Stage 00", "Stage 01", "Stage 02", "Stage 03", "Stage 04", "Stage 05", "Stage 06", "Visibility"];

stages.forEach(stage => makeTab(uiTabs, stage));


const panelWrap = createDiv();
```

```
panelWrap.parent(uiDrawer);
```

```
stages.forEach(stage => {
```

```
    stagePanels[stage] = createDiv();
```

```
    stagePanels[stage].parent(panelWrap);
```

```
    stagePanels[stage].style("display", stage === activeStage ? "grid" : "none");
```

```
    stagePanels[stage].style("grid-template-columns", "repeat(auto-fit, minmax(220px, 1fr))");
```

```
    stagePanels[stage].style("gap", "8px");
```

```
});
```

```
buildStageControls();
```

```
hudPanel = createDiv();
```

```
hudPanel.id("urban-matrix-hud");
```

```
hudPanel.style("position", "fixed");
```

```
hudPanel.style("top", "12px");
```

```
hudPanel.style("right", "12px");
```

```
hudPanel.style("z-index", "10");
```

```
hudPanel.style("max-width", "420px");
```

```
hudPanel.style("padding", "12px 14px");
```

```
hudPanel.style("border", "1px solid rgba(157,220,127,0.8)");
```

```
hudPanel.style("border-radius", "10px");
```

```
hudPanel.style("background", "rgba(20,22,22,0.68)");
```

```
hudPanel.style("backdrop-filter", "blur(8px)");
```

```
hudPanel.style("color", "white");
```

```
hudPanel.style("font-family", "Inter, Arial, sans-serif");
```

```
hudPanel.style("font-size", "12px");
```

```
hudPanel.style("line-height", "1.38");
```

```
}
```

```
function makeButton(parent, label, fn) {
```

```
  const b = createButton(label);
```

```
  b.parent(parent);
```

```
  b.mousePressed(fn);
```

```
  b.style("background", "#202020");
```

```
  b.style("color", "white");
```

```
  b.style("border", "1px solid #9ddc7f");
```

```
  b.style("border-radius", "6px");
```

```
  b.style("padding", "6px 9px");
```

```
  b.style("font-size", "11px");
```

```
  b.style("cursor", "pointer");
```

```
  return b;
```

```
}
```

```
function addToolbarSelect(parent, label, key, options, selected, changedFn) {
```

```
  const wrap = createSpan();
```

```
  wrap.parent(parent);
```

```
  wrap.style("display", "inline-flex");
```

```
  wrap.style("align-items", "center");
```

```
  wrap.style("gap", "4px");
```

```
  const l = createSpan(label);
```

```
  l.parent(wrap);
```

```
  l.style("font-size", "11px");
```

```
  l.style("color", "#c8f7a6");
```

```
const s = createSelect();

s.parent(wrap);

options.forEach(o => s.option(o));

s.selected(selected);

s.style("font-size", "11px");

if (changedFn) s.changed(changedFn);

selects[key] = s;

}
```

```
function makeTab(parent, stage) {

  const b = createButton(stage);

  b.parent(parent);

  b.mousePressed(() => setActiveStage(stage));

  b.style("background", stage === activeStage ? "#355530" : "#222");

  b.style("color", "white");

  b.style("border", "1px solid #555");

  b.style("border-radius", "6px");

  b.style("padding", "6px 9px");

  b.style("font-size", "11px");

  b.attribute("data-stage-tab", stage);

}
```

```
function setActiveStage(stage) {

  activeStage = stage;

  for (let k in stagePanels) stagePanels[k].style("display", k === stage ? "grid" : "none");

  const tabs = uiTabs.elt.querySelectorAll("button");

  tabs.forEach(t => {

    t.style.background = t.getAttribute("data-stage-tab") === stage ? "#355530" : "#222";
```

```
});
```

```
}
```

```
function buildStageControls() {
```

```
  addSlider("Stage 00", "siteWidth", "Site width m", 60, 220, 130, 1);
```

```
  addSlider("Stage 00", "siteDepth", "Site depth m", 50, 180, 95, 1);
```

```
  addSlider("Stage 00", "frontSetback", "Front setback m", 2, 20, 8, 0.5);
```

```
  addSlider("Stage 00", "rearSetback", "Rear setback m", 2, 20, 7, 0.5);
```

```
  addSlider("Stage 00", "leftSetback", "Left setback m", 2, 18, 6, 0.5);
```

```
  addSlider("Stage 00", "rightSetback", "Right setback m", 2, 18, 6, 0.5);
```

```
  addSlider("Stage 00", "roadWidth", "Road width m", 5, 18, 8, 0.5);
```

```
  addSlider("Stage 00", "footpathWidth", "Footpath m", 1.5, 7, 3, 0.5);
```

```
  addSlider("Stage 00", "landscapeBuffer", "Landscape m", 1, 12, 4, 0.5);
```

```
  addSlider("Stage 00", "gridSpacing", "Planning grid m", 2, 10, 5, 1);
```

```
  addSlider("Stage 01", "moduleW", "Module W m", 4.5, 12, 7.2, 0.1);
```

```
  addSlider("Stage 01", "moduleD", "Module D m", 5.5, 14, 9, 0.1);
```

```
  addSlider("Stage 01", "moduleH", "Floor H m", 2.8, 4.2, 3.2, 0.1);
```

```
  addSlider("Stage 01", "moduleGap", "Module gap m", 0.6, 3.8, 1.2, 0.1);
```

```
  addSlider("Stage 01", "cols", "Columns", 3, 16, 8, 1);
```

```
  addSlider("Stage 01", "rows", "Rows", 2, 10, 5, 1);
```

```
  addSlider("Stage 01", "floors", "Floors", 1, 12, 6, 1);
```

```
  addSlider("Stage 01", "density", "Density", 0.25, 1.0, 0.78, 0.01);
```

```
  addSlider("Stage 01", "setback", "Floor setback m", 0, 8, 2.2, 0.1);
```

```
  addSlider("Stage 01", "stagger", "Stagger m", 0, 2.5, 0.8, 0.1);
```

```
  addSlider("Stage 01", "voidStrength", "Terrace voids", 0, 1, 0.34, 0.01);
```

```
  addSlider("Stage 01", "courtyard", "Courtyard", 0, 1, 0.42, 0.01);
```

```
  addSlider("Stage 01", "zoom", "Model scale", 0.45, 1.35, 0.74, 0.01);
```

addSlider("Stage 02", "balconyDepth", "Balcony depth m", 0, 4, 1.8, 0.1);

addSlider("Stage 02", "railingHeight", "Railing H m", 0.8, 1.4, 1.1, 0.05);

addSlider("Stage 02", "slabThickness", "Slab m", 0.2, 0.8, 0.35, 0.05);

addSlider("Stage 02", "windowScale", "Window %", 0.2, 1, 0.52, 0.01);

addSlider("Stage 02", "facadeDetail", "Facade detail", 0, 1, 0.72, 0.01);

addSlider("Stage 02", "screenAmount", "Screens", 0, 1, 0.58, 0.01);

addSlider("Stage 02", "finCount", "Fin count", 2, 12, 7, 1);

addSlider("Stage 02", "finDepth", "Fin depth m", 0.1, 1.2, 0.45, 0.05);

addSlider("Stage 02", "louvreCount", "Louvres", 2, 10, 6, 1);

addSlider("Stage 02", "jaaliCols", "Jaali cols", 3, 9, 6, 1);

addSlider("Stage 02", "jaaliRows", "Jaali rows", 3, 8, 5, 1);

addSlider("Stage 02", "parapetHeight", "Parapet m", 0, 1.6, 0.9, 0.05);

addSlider("Stage 02", "coreSize", "Core size m", 3, 8, 4.8, 0.1);

addSlider("Stage 03", "sunAzimuth", "Sun direction", 0, 360, 135, 1);

addSlider("Stage 03", "sunAltitude", "Sun altitude", 5, 80, 38, 1);

addSlider("Stage 03", "shadowLength", "Shadow", 0, 1, 0.45, 0.01);

addSlider("Stage 03", "shadeResponse", "Shade", 0, 1, 0.65, 0.01);

addSlider("Stage 03", "passiveFacade", "Ventilation", 0, 1, 0.72, 0.01);

addSlider("Stage 03", "greenAmount", "Green roof", 0, 1, 0.68, 0.01);

addSlider("Stage 03", "vegetationSize", "Vegetation", 0.3, 1.5, 0.85, 0.01);

addSlider("Stage 03", "treeCount", "Trees", 4, 24, 12, 1);

addSlider("Stage 03", "publicSpaceScale", "Public scale", 0.4, 1.4, 0.95, 0.01);

addSlider("Stage 03", "analysisOverlay", "Overlay", 0, 1, 0.55, 0.01);

addSlider("Stage 04", "codeStrictness", "Code logic", 0, 1, 0.70, 0.01);

addSlider("Stage 04", "accessibility", "Access", 0, 1, 0.72, 0.01);

addSlider("Stage 04", "fireSafety", "Fire logic", 0, 1, 0.65, 0.01);

addSlider("Stage 04", "energyTarget", "Energy", 0, 1, 0.76, 0.01);

addSlider("Stage 04", "carbonTarget", "Carbon", 0, 1, 0.70, 0.01);

addSlider("Stage 04", "renewableArea", "PV area", 0, 1, 0.45, 0.01);

addSlider("Stage 04", "waterReuse", "Water", 0, 1, 0.62, 0.01);

addSlider("Stage 04", "ugfTarget", "Greening", 0, 1, 0.72, 0.01);

addSlider("Stage 04", "comfortTarget", "Comfort", 0, 1, 0.70, 0.01);

addSlider("Stage 05", "exportResolution", "Export x", 1, 5, 3, 1);

addSlider("Stage 05", "exportScale", "Export scale", 0.5, 2, 1, 0.01);

addSlider("Stage 06", "lineWeight", "Line weight", 0.5, 4, 1.2, 0.1);

addSlider("Stage 06", "textSize2D", "Text size", 8, 24, 12, 1);

addSlider("Stage 06", "dimensionPower", "Dimension power", 0, 1, 0.90, 0.01);

addToggle("Visibility", "stage01", "Show Massing", true);

addToggle("Visibility", "stage02", "Show Details", true);

addToggle("Visibility", "stage03", "Show Environment", true);

addToggle("Visibility", "stage04", "Show Performance", true);

addToggle("Visibility", "siteBoundary", "Site Boundary A/B/C/D", true);

addToggle("Visibility", "dimensions", "Dimensions", true);

addToggle("Visibility", "labels", "Labels", true);

addToggle("Visibility", "context", "Context Buildings", true);

addToggle("Visibility", "core", "Lift/Stair Core", true);

addToggle("Visibility", "pv", "PV Panels", true);

addToggle("Visibility", "publicRealm", "Public Realm", true);

addToggle("Visibility", "autoFit", "Auto-fit Inside Site", true);

}

```
function addSlider(stage, key, label, minVal, maxVal, startVal, stepVal) {
```

```
  const wrap = createDiv();
```

```
  wrap.parent(stagePanels[stage]);
```

```
  wrap.style("display", "grid");
```

```
  wrap.style("grid-template-columns", "95px 1fr 46px");
```

```
  wrap.style("gap", "6px");
```

```
  wrap.style("align-items", "center");
```

```
  wrap.style("padding", "6px");
```

```
  wrap.style("background", "rgba(255,255,255,0.04)");
```

```
  wrap.style("border", "1px solid rgba(255,255,255,0.08)");
```

```
  wrap.style("border-radius", "6px");
```

```
  const l = createSpan(label);
```

```
  l.parent(wrap);
```

```
  l.style("font-size", "10px");
```

```
  l.style("color", "#eeeeee");
```

```
  const s = createSlider(minVal, maxVal, startVal, stepVal);
```

```
  s.parent(wrap);
```

```
  s.style("width", "100%");
```

```
  const v = createSpan(String(startVal));
```

```
  v.parent(wrap);
```

```
  v.style("font-size", "10px");
```

```
  v.style("color", "#c8f7a6");
```

```
  v.style("text-align", "right");
```

```
controls[key] = s;
```

```
labels[key] = v;
```

```
}
```

```
function addToggle(stage, key, label, checkedValue) {
```

```
  const cb = createCheckbox(label, checkedValue);
```

```
  cb.parent(stagePanels[stage]);
```

```
  cb.style("font-size", "11px");
```

```
  cb.style("color", "white");
```

```
  cb.style("padding", "7px");
```

```
  cb.style("background", "rgba(255,255,255,0.04)");
```

```
  cb.style("border-radius", "6px");
```

```
  toggles[key] = cb;
```

```
}
```

```
function updateLabels() {
```

```
  for (const k in controls) {
```

```
    const v = Number(controls[k].value());
```

```
    const decimals = Math.abs(v - Math.round(v)) < 0.001 ? 0 : 2;
```

```
    labels[k].html(nf(v, 1, decimals));
```

```
  }
```

```
}
```

```
function value(key) {
```

```
  return controls[key] ? Number(controls[key].value()) : 0;
```

```
}
```

```
function checked(key) {
```

```
return toggles[key] ? toggles[key].checked() : false;
```

```
}
```

```
function toggleDrawer() {
```

```
    drawerOpen = !drawerOpen;
```

```
    uiDrawer.style("display", drawerOpen ? "block" : "none");
```

```
    buttons.collapse.html(drawerOpen ? "Collapse Drawer" : "Open Drawer");
```

```
}
```

```
function toggleUIVisibility() {
```

```
    uiVisible = !uiVisible;
```

```
    uiRoot.style("display", uiVisible ? "block" : "none");
```

```
}
```

```
function toggleCleanMode() {
```

```
    cleanMode = !cleanMode;
```

```
    hudPanel.style("display", cleanMode ? "none" : "block");
```

```
    uiRoot.style("opacity", cleanMode ? "0" : "1");
```

```
    uiRoot.style("pointer-events", cleanMode ? "none" : "auto");
```

```
}
```

```
/* =====
```

```
METRIC AND SITE HELPERS
```

```
===== */
```

```
function mToP5(m) { return m / P5_TO_METRE; }
```

```
function p5ToM(p) { return p * P5_TO_METRE; }
```

```
function fmtM(m) { return nf(m, 1, 1) + " m"; }
```

```
function safeStep(step, fallback) {  
  
  const s = Number(step);  
  
  if (!isFinite(s) || s <= 0) return fallback;  
  
  return max(s, fallback);  
  
}
```

```
function getSite() {  
  
  return {  
  
    wM: value("siteWidth"),  
  
    dM: value("siteDepth"),  
  
    w: mToP5(value("siteWidth")),  
  
    d: mToP5(value("siteDepth")),  
  
    frontM: value("frontSetback"),  
  
    rearM: value("rearSetback"),  
  
    leftM: value("leftSetback"),  
  
    rightM: value("rightSetback"),  
  
    front: mToP5(value("frontSetback")),  
  
    rear: mToP5(value("rearSetback")),  
  
    left: mToP5(value("leftSetback")),  
  
    right: mToP5(value("rightSetback")),  
  
    roadW: mToP5(value("roadWidth")),  
  
    footpathW: mToP5(value("footpathWidth")),  
  
    buffer: mToP5(value("landscapeBuffer"))  
  
  };  
  
}
```

```
function getBuildableEnvelope() {
```

```
const s = getSite();
```

```
const x = s.roadW * 0.35;
```

```
const z = 0;
```

```
const w = max(80, s.w - s.left - s.right - s.roadW - s.footpathW - s.buffer);
```

```
const d = max(80, s.d - s.front - s.rear - s.buffer * 2);
```

```
return { x, z, w, d };
```

```
}
```

```
function getSafeGap() {
```

```
return max(mToP5(value("moduleGap")), mToP5(0.8 + value("codeStrictness") * 0.8));
```

```
}
```

```
function getEffectiveGrid() {
```

```
const env = getBuildableEnvelope();
```

```
const moduleW = mToP5(value("moduleW"));
```

```
const moduleD = mToP5(value("moduleD"));
```

```
const gap = getSafeGap();
```

```
const maxCols = max(1, floor(env.w / (moduleW + gap)));
```

```
const maxRows = max(1, floor(env.d / (moduleD + gap)));
```

```
let cols = value("cols");
```

```
let rows = value("rows");
```

```
if (checked("autoFit")) {
```

```
cols = min(cols, maxCols);
```

```
rows = min(rows, maxRows);
```

```
}
```

```
if (value("cols") > maxCols || value("rows") > maxRows) boundaryWarning = true;
```

```
return { cols, rows, maxCols, maxRows };
```

```
}
```

```
/* =====
```

CAMERA AND LIVE VIEW SWITCHING

```
===== */
```

```
function updateCameraForDrawingView(view) {
```

```
    const env = getBuildableEnvelope();
```

```
    const h = mToP5(value("moduleH")) * value("floors");
```

```
    const centreX = env.x;
```

```
    const centreZ = env.z;
```

```
    const dist = max(env.w, env.d, h) * 1.35;
```

```
    if (view === "Current 3D" || view === "Moodboard" || view === "Technical Sheet") {
```

```
        perspective();
```

```
        applyOrbitCamera();
```

```
        return;
```

```
    }
```

```
    ortho(-width / 2, width / 2, -height / 2, height / 2, -6000, 6000);
```

```
    if (view.includes("Plan") || view === "Site Plan" || view === "Masterplan") {
```

```
        camera(centreX, -dist * 1.35, centreZ, centreX, 0, centreZ, 0, 0, -1);
```

```
        return;
```

```
    }
```

```
    if (view === "Front Elevation") {
```

```
camera(centreX, -h / 2, centreZ - dist, centreX, -h / 2, centreZ, 0, 1, 0);
```

```
return;
```

```
}
```

```
if (view === "Rear Elevation") {
```

```
camera(centreX, -h / 2, centreZ + dist, centreX, -h / 2, centreZ, 0, 1, 0);
```

```
return;
```

```
}
```

```
if (view === "Left Elevation") {
```

```
camera(centreX - dist, -h / 2, centreZ, centreX, -h / 2, centreZ, 0, 1, 0);
```

```
return;
```

```
}
```

```
if (view === "Right Elevation") {
```

```
camera(centreX + dist, -h / 2, centreZ, centreX, -h / 2, centreZ, 0, 1, 0);
```

```
return;
```

```
}
```

```
if (view.includes("Section")) {
```

```
camera(centreX, -h / 2, centreZ - dist, centreX, -h / 2, centreZ, 0, 1, 0);
```

```
return;
```

```
}
```

```
applyOrbitCamera();
```

```
}
```

```
function applyOrbitCamera() {
```

```
const x = camTarget.x + camDist * cos(camPitch) * sin(camYaw);
```

```
const y = camTarget.y + camDist * sin(camPitch);
```

```
const z = camTarget.z + camDist * cos(camPitch) * cos(camYaw);
```

```
camera(x, y, z, camTarget.x, camTarget.y, camTarget.z, 0, 1, 0);
```

```
}
```

```
function mousePressed() {
```

```
  if (mouseY >= 0 && mouseY <= height) {
```

```
    dragging = true;
```

```
    lastMouseX = mouseX;
```

```
    lastMouseY = mouseY;
```

```
  }
```

```
}
```

```
function mouseReleased() {
```

```
  dragging = false;
```

```
}
```

```
function mouseDragged() {
```

```
  if (!dragging || selects.drawingType.value() !== "Current 3D") return;
```

```
  const dx = mouseX - lastMouseX;
```

```
  const dy = mouseY - lastMouseY;
```

```
  if (mouseButton === LEFT) {
```

```
    camYaw += dx * 0.006;
```

```
    camPitch = constrain(camPitch + dy * 0.005, -1.35, -0.08);
```

```
  } else if (mouseButton === RIGHT) {
```

```
camTarget.x -= dx * 0.72;
```

```
camTarget.z -= dy * 0.72;
```

```
}
```

```
lastMouseX = mouseX;
```

```
lastMouseY = mouseY;
```

```
}
```

```
function mouseWheel(event) {
```

```
  if (selects.drawingType.value() === "Current 3D") {
```

```
    camDist = constrain(camDist + event.delta * 0.65, 260, 3200);
```

```
    return false;
```

```
  }
```

```
}
```

```
/* =====
```

```
  3D SITE, MODEL AND DETAILS
```

```
===== */
```

```
function drawSiteSystem3D() {
```

```
  const s = getSite();
```

```
  const env = getBuildableEnvelope();
```

```
  push();
```

```
  translate(0, groundY + 6, 0);
```

```
  noStroke();
```

```
  fill(...palette.ground);
```

```
  box(s.w, 12, s.d);
```

```
fill(...palette.road);
```

```
push();
```

```
translate(-s.w / 2 + s.roadW / 2, -8, 0);
```

```
box(s.roadW, 8, s.d);
```

```
pop();
```

```
fill(...palette.footpath);
```

```
push();
```

```
translate(-s.w / 2 + s.roadW + s.footpathW / 2, -10, 0);
```

```
box(s.footpathW, 6, s.d);
```

```
pop();
```

```
pop();
```

```
if (checked("siteBoundary") && !cleanMode) {
```

```
    drawBoundaryBox3D(s);
```

```
    drawBuildableEnvelope3D(env);
```

```
    drawPlanningGrid3D(s);
```

```
}
```

```
}
```

```
function drawBoundaryBox3D(s) {
```

```
    push();
```

```
    translate(0, groundY - 5, 0);
```

```
    noFill();
```

```
    stroke(...palette.boundary, 210);
```

```
    strokeWeight(2);
```

```
    box(s.w, 2, s.d);
```

```
pop();
```

```
}
```

```
function drawBuildableEnvelope3D(env) {
```

```
  push();
```

```
  translate(env.x, groundY - 8, env.z);
```

```
  noFill();
```

```
  stroke(...palette.envelope, 200);
```

```
  strokeWeight(2);
```

```
  box(env.w, 2, env.d);
```

```
  pop();
```

```
}
```

```
function drawPlanningGrid3D(s) {
```

```
  let step = safeStep(mToP5(value("gridSpacing")), 20);
```

```
  const maxLines = 140;
```

```
  let count = 0;
```

```
  stroke(110, 110, 100, 40);
```

```
  strokeWeight(1);
```

```
  for (let x = -s.w / 2; x <= s.w / 2 && count < maxLines; x += step, count++) {
```

```
    line(x, groundY - 9, -s.d / 2, x, groundY - 9, s.d / 2);
```

```
  }
```

```
  count = 0;
```

```
  for (let z = -s.d / 2; z <= s.d / 2 && count < maxLines; z += step, count++) {
```

```
line(-s.w / 2, groundY - 9, z, s.w / 2, groundY - 9, z);
```

```
}
```

```
noStroke();
```

```
}
```

```
function drawModularHousing3D() {
```

```
noiseSeed(seedValue);
```

```
const moduleW = mToP5(value("moduleW"));
```

```
const moduleD = mToP5(value("moduleD"));
```

```
const moduleH = mToP5(value("moduleH"));
```

```
const gap = getSafeGap();
```

```
const grid = getEffectiveGrid();
```

```
const floors = value("floors");
```

```
const env = getBuildableEnvelope();
```

```
const totalW = grid.cols * (moduleW + gap);
```

```
const totalD = grid.rows * (moduleD + gap);
```

```
const setback = mToP5(value("setback"));
```

```
const stagger = min(mToP5(value("stagger")), gap * 0.85);
```

```
for (let f = 0; f < floors; f++) {
```

```
  for (let r = 0; r < grid.rows; r++) {
```

```
    for (let c = 0; c < grid.cols; c++) {
```

```
      if (!shouldPlaceModule(c, r, f, grid.cols, grid.rows, floors)) continue;
```

```
      const pos = modulePosition(
```

```
        c, r, f, grid.cols, grid.rows,
```

```
moduleW, moduleD, moduleH, gap,
```

```
totalW, totalD, setback, stagger,
```

```
env.x, env.z
```

```
);
```

```
generatedModules.push(makeModuleData(c, r, f, pos, moduleW, moduleD, moduleH));
```

```
drawModule3D(pos.x, pos.y, pos.z, moduleW, moduleH, moduleD, c, r, f, grid.cols, grid.rows);
```

```
}
```

```
}
```

```
}
```

```
if (checked("core")) drawCore3D(env, totalW, totalD, moduleH, floors);
```

```
}
```

```
function shouldPlaceModule(c, r, f, cols, rows, floors) {
```

```
const edge = c === 0 || c === cols - 1 || r === 0 || r === rows - 1;
```

```
const lowerBase = f <= 1;
```

```
const centreC = (cols - 1) / 2;
```

```
const centreR = (rows - 1) / 2;
```

```
const dc = abs(c - centreC) / max(1, cols / 2);
```

```
const dr = abs(r - centreR) / max(1, rows / 2);
```

```
const centreDistance = sqrt(dc * dc + dr * dr);
```

```
const courtyardVoid = centreDistance < value("courtyard") * 0.52 && f < floors - 1;
```

```
const noiseValue = noise(c * 0.42 + seedValue * 0.01, r * 0.53, f * 0.36);
```

```
const terraceCut = noiseValue > value("density");
```

```
const upperFloorVoid = f > 1 && hash01(c, r, f, 10) < value("voidStrength") * 0.18;
```

```
if (lowerBase && edge) return true;
```

```
if (courtyardVoid && !edge) return false;
```

```
if (upperFloorVoid && !edge) return false;
```

```
if (terraceCut && !edge && !lowerBase) return false;
```

```
return true;
```

```
}
```

```
function modulePosition(c, r, f, cols, rows, w, d, h, gap, totalW, totalD, setback, stagger, shiftX, shiftZ) {
```

```
let x = c * (w + gap) - totalW / 2 + w / 2 + shiftX;
```

```
let y = groundY - h / 2 - f * h;
```

```
let z = r * (d + gap) - totalD / 2 + d / 2 + shiftZ + f * setback;
```

```
x += sin(c * 1.7 + r * 0.9 + f * 1.15 + seedValue) * stagger;
```

```
z += cos(c * 0.8 + r * 1.4 + f * 1.05 + seedValue) * stagger * 0.30;
```

```
return createVector(x, y, z);
```

```
}
```

```
function makeModuleData(c, r, f, pos, w, d, h) {
```

```
return {
```

```
id: `M_${c}_${r}_${f}`,
```

```
column: c,
```

```
row: r,
```

```
floor: f,
```

```
x: pos.x,
```

```
y: pos.y,
```

```
    z: pos.z,  
    width: w,  
    depth: d,  
    height: h,  
    widthM: value("moduleW"),  
    depthM: value("moduleD"),  
    heightM: value("moduleH"),  
    materialSystem: selects.materialSystem.value(),  
    type: ["living", "bedroom", "terrace", "screened", "corner"][(c + r + f) % 5]  
  };  
}
```

```
function drawModule3D(x, y, z, w, h, d, c, r, f, cols, rows) {
```

```
  push();  
  translate(x, y, z);  
  noStroke();  
  applyModuleFill(c, r, f);  
  box(w * 0.96, h * 0.94, d * 0.96);  
  pop();
```

```
  if (checked("stage02")) {  
    drawRoofAndParapet3D(x, y, z, w, h, d);  
    drawFacadeSides3D(x, y, z, w, h, d, c, r, f, cols, rows);  
  }
```

```
  if (checked("stage03") && hash01(c, r, f, 21) < value("greenAmount")) {  
    drawGreenRoof3D(x, y - h / 2 - 8, z, w, d, c, r, f);  
  }
```

```
}
```

```
function applyModuleFill(c, r, f) {
```

```
  if (selects.viewMode.value() === "Material Diagram") {
```

```
    const mat = materialProfiles[selects.materialSystem.value()];
```

```
    fill(...mat.colour);
```

```
  } else {
```

```
    const arr = [palette.concrete, palette.clay, palette.stone, palette.brick, palette.timber][(c + r + f) % 5];
```

```
    fill(...arr);
```

```
  }
```

```
}
```

```
function drawRoofAndParapet3D(x, y, z, w, h, d) {
```

```
  const slab = mToP5(value("slabThickness"));
```

```
  const parapet = mToP5(value("parapetHeight"));
```

```
  push();
```

```
  translate(x, y - h / 2 - slab / 2, z);
```

```
  fill(225, 215, 198);
```

```
  noStroke();
```

```
  box(w * 1.02, slab, d * 1.02);
```

```
  pop();
```

```
  if (parapet > 1) {
```

```
    push();
```

```
    translate(x, y - h / 2 - slab - parapet / 2, z);
```

```
    fill(210, 197, 177);
```

```
    box(w, parapet, 4);
```

```
    pop();
```

```
  }
```

```
}
```

```
function drawFacadeSides3D(x, y, z, w, h, d, c, r, f, cols, rows) {
```

```
  const sides = [
```

```
    { name: "front", outer: r === 0, tx: 0, tz: -d / 2 - 4, rot: 0, faceW: w },
```

```
    { name: "rear", outer: r === rows - 1, tx: 0, tz: d / 2 + 4, rot: PI, faceW: w },
```

```
    { name: "left", outer: c === 0, tx: -w / 2 - 4, tz: 0, rot: -HALF_PI, faceW: d },
```

```
    { name: "right", outer: c === cols - 1, tx: w / 2 + 4, tz: 0, rot: HALF_PI, faceW: d }
```

```
  ];
```

```
  for (const side of sides) {
```

```
    if (!side.outer && hash01(c, r, f, side.name.length) > 0.2) continue;
```

```
    push();
```

```
    translate(x + side.tx, y, z + side.tz);
```

```
    rotateY(side.rot);
```

```
    drawOpening3D(side.faceW, h, c, r, f);
```

```
    if (side.name === "front" && value("balconyDepth") > 0.2) drawBalcony3D(side.faceW, h);
```

```
    pop();
```

```
  }
```

```
}
```

```
function drawOpening3D(w, h, c, r, f) {
```

```
  fill(...palette.glass, 190);
```

```
  noStroke();
```

```
  box(w * value("windowScale"), h * 0.45, 4);
```

```
if (hash01(c, r, f, 61) < value("screenAmount")) drawScreen3D(w, h, c, r, f);  
}
```

```
function drawScreen3D(w, h, c, r, f) {
```

```
  const count = value("finCount");
```

```
  const depth = mToP5(value("finDepth"));
```

```
  fill(...palette.timber);
```

```
  for (let i = 0; i < count; i++) {
```

```
    const px = map(i, 0, max(1, count - 1), -w * 0.35, w * 0.35);
```

```
    push();
```

```
    translate(px, 0, -6);
```

```
    box(4, h * 0.72, depth);
```

```
    pop();
```

```
  }
```

```
}
```

```
function drawBalcony3D(w, h) {
```

```
  const depth = mToP5(value("balconyDepth"));
```

```
  const rail = mToP5(value("railingHeight"));
```

```
  push();
```

```
  translate(0, h * 0.25, -depth / 2);
```

```
  fill(210, 176, 132);
```

```
  box(w * 0.78, 4, depth);
```

```
  fill(...palette.rail);
```

```
translate(0, -rail, -depth / 2);
```

```
box(w * 0.78, 4, 4);
```

```
pop();
```

```
}
```

```
function drawGreenRoof3D(x, y, z, w, d, c, r, f) {
```

```
  push();
```

```
  translate(x, y, z);
```

```
  fill(...palette.grass);
```

```
  box(w * 0.76, 5, d * 0.62);
```

```
  fill(...palette.shrub);
```

```
  for (let i = 0; i < 4; i++) {
```

```
    push();
```

```
    translate(
```

```
      map(hash01(c, r, f, i + 51), 0, 1, -w * 0.26, w * 0.26),
```

```
      -8,
```

```
      map(hash01(c, r, f, i + 71), 0, 1, -d * 0.20, d * 0.20)
```

```
    );
```

```
    sphere(4 * value("vegetationSize"), 6, 4);
```

```
    pop();
```

```
  }
```

```
  pop();
```

```
}
```

```
function drawCore3D(env, totalW, totalD, moduleH, floors) {
```

```
const core = mToP5(value("coreSize"));
```

```
push();
```

```
translate(env.x - totalW / 2 - core * 0.55, groundY - (moduleH * floors) / 2, env.z);
```

```
fill(188, 174, 148);
```

```
box(core, moduleH * floors, core);
```

```
pop();
```

```
}
```

```
function drawPublicRealm3D() {
```

```
const s = getSite();
```

```
drawPlaza3D(s.w * 0.23, groundY - 4, s.d * 0.30, value("publicSpaceScale"));
```

```
drawPark3D(-s.w * 0.18, groundY - 4, s.d * 0.31, value("publicSpaceScale"));
```

```
}
```

```
function drawPlaza3D(x, y, z, sc) {
```

```
push();
```

```
translate(x, y, z);
```

```
scale(sc);
```

```
fill(236, 220, 198);
```

```
noStroke();
```

```
box(260, 5, 160);
```

```
pop();
```

```
}
```

```
function drawPark3D(x, y, z, sc) {
```

```
push();
```

```
translate(x, y, z);
```

```
scale(sc);
```

```
fill(...palette.grass);
```

```
noStroke();
```

```
box(250, 5, 145);
```

```
drawTree3D(-80, -28, -35, 0.8);
```

```
drawTree3D(70, -28, 35, 0.75);
```

```
pop();
```

```
}
```

```
function drawContextBuildings3D() {
```

```
  const s = getSite();
```

```
  fill(198, 194, 182, 155);
```

```
  noStroke();
```

```
  push();
```

```
  translate(s.w / 2 + 140, groundY - 60, -s.d * 0.25);
```

```
  box(250, 120, 150);
```

```
  pop();
```

```
  push();
```

```
  translate(s.w / 2 + 160, groundY - 55, s.d * 0.24);
```

```
  box(210, 110, 190);
```

```
  pop();
```

```
}
```

```
function drawTree3D(x, y, z, sc) {
```

```
  push();
```

```
translate(x, y, z);
```

```
scale(sc * value("vegetationSize"));
```

```
fill(...palette.trunk);
```

```
cylinder(6, 60, 6, 1);
```

```
translate(0, -48, 0);
```

```
fill(...palette.grass);
```

```
sphere(28, 7, 5);
```

```
pop();
```

```
}
```

```
function drawEnvironmental3D() {
```

```
  if (cleanMode) return;
```

```
  const overlay = value("analysisOverlay");
```

```
  const az = radians(value("sunAzimuth"));
```

```
  const alt = radians(value("sunAltitude"));
```

```
  const sx = cos(az) * 1250;
```

```
  const sz = sin(az) * 1250;
```

```
  const sy = -sin(alt) * 700 - 420;
```

```
  push();
```

```
  translate(sx, sy, sz);
```

```
  fill(...palette.pv, 220 * overlay);
```

```
  sphere(36, 14, 8);
```

```
  pop();
```

```
  stroke(255, 220, 140, 130 * overlay);
```

```
  line(sx, sy, sz, getBuildableEnvelope().x, -260, getBuildableEnvelope().z);
```

```
noStroke();
```

```
}
```

```
function drawPerformanceOverlays3D() {
```

```
  if (checked("pv")) drawPVPanels3D();
```

```
  drawWaterReuse3D();
```

```
}
```

```
function drawPVPanels3D() {
```

```
  const pv = value("renewableArea");
```

```
  if (pv < 0.05) return;
```

```
  const env = getBuildableEnvelope();
```

```
  const h = mToP5(value("moduleH")) * value("floors");
```

```
  const count = floor(map(pv, 0, 1, 2, 14));
```

```
  fill(...palette.pv, 210);
```

```
  noStroke();
```

```
  for (let i = 0; i < count; i++) {
```

```
    push();
```

```
    translate(
```

```
      env.x - env.w * 0.24 + (i % 5) * 58,
```

```
      groundY - h - 20,
```

```
      env.z - env.d * 0.32 + floor(i / 5) * 44
```

```
    );
```

```
    rotateX(0.08);
```

```
    box(52, 4, 34);
```

```
pop();
```

```
}
```

```
}
```

```
function drawWaterReuse3D() {
```

```
  if (value("waterReuse") < 0.1) return;
```

```
  const s = getSite();
```

```
  push();
```

```
  translate(s.w / 2 - 90, groundY - 20, -s.d / 2 + 160);
```

```
  fill(...palette.water, 160);
```

```
  cylinder(28, 42, 12, 1);
```

```
  pop();
```

```
}
```

```
/* =====
```

PERFORMANCE, PRESETS AND ITERATIONS

```
===== */
```

```
function getCurrentRegion() {
```

```
  return regionProfiles[selects.region.value()];
```

```
}
```

```
function getCurrentMaterial() {
```

```
  return materialProfiles[selects.materialSystem.value()];
```

```
}
```

```
function calculateOverallPerformance() {
```

```
  const region = getCurrentRegion();
```

```
  const mat = getCurrentMaterial();
```

```
  const energy = clamp01(
```

```
    value("energyTarget") * 0.38 +
```

```
    value("passiveFacade") * region.ventilationBias * 0.22 +
```

```
    value("shadeResponse") * region.shadingBias * 0.18 +
```

```
    value("renewableArea") * region.pvBias * 0.22
```

```
  );
```

```
  const carbon = clamp01(
```

```
    value("carbonTarget") * 0.38 +
```

```
    (1 - mat.carbonFactor) * 0.26 +
```

```
    value("renewableArea") * 0.18 +
```

```
    value("density") * 0.18
```

```
  );
```

```
  const greening = clamp01(
```

```
    value("greenAmount") * 0.38 +
```

```
    value("ugfTarget") * 0.34 +
```

```
    value("treeCount") / 24 * 0.18 +
```

```
    value("waterReuse") * 0.10
```

```
  );
```

```
  const access = clamp01(
```

```
    value("accessibility") * 0.55 +
```

```
    value("publicSpaceScale") / 1.4 * 0.25 +
```

```
value("codeStrictness") * 0.20
```

```
);
```

```
const comfort = clamp01(
```

```
value("comfortTarget") * 0.38 +
```

```
value("courtyard") * 0.18 +
```

```
value("shadeResponse") * 0.20 +
```

```
value("passiveFacade") * 0.14 +
```

```
value("greenAmount") * 0.10
```

```
);
```

```
const fire = clamp01(
```

```
value("fireSafety") * 0.55 +
```

```
value("codeStrictness") * 0.25 +
```

```
(1.0 - value("facadeDetail") * 0.2) * 0.20
```

```
);
```

```
return {
```

```
energy,
```

```
carbon,
```

```
greening,
```

```
access,
```

```
comfort,
```

```
fire,
```

```
total: clamp01((energy + carbon + greening + access + comfort + fire) / 6)
```

```
};
```

```
}
```

```
function clamp01(v) {
```

```
  return constrain(v, 0, 1);
```

```
}
```

```
function updateMetrics() {
```

```
  generatedMetrics = calculateOverallPerformance();
```

```
}
```

```
function applyRegionPreset() {
```

```
  const r = getCurrentRegion();
```

```
  setValue("shadeResponse", r.shadingBias);
```

```
  setValue("passiveFacade", r.ventilationBias);
```

```
  setValue("greenAmount", r.waterBias * 0.75);
```

```
  setValue("waterReuse", r.waterBias);
```

```
  setValue("renewableArea", r.pvBias);
```

```
  setValue("sunAltitude", map(r.sunBias, 0, 1, 25, 62));
```

```
  setValue("parapetHeight", map(r.plinthBias, 0, 1, 0.6, 1.4));
```

```
  selects.materialSystem.selected(r.recommendedMaterial);
```

```
}
```

```
function applyOptimisationPreset() {
```

```
  const m = selects.optimisationMode.value();
```

```
  if (m === "Balanced") {
```

```
    setValue("energyTarget", 0.72);
```

```
    setValue("carbonTarget", 0.68);
```

```
    setValue("ugfTarget", 0.70);
```

```
setValue("comfortTarget", 0.70);
```

```
}
```

```
if (m === "Maximum Performance") {
```

```
setValue("energyTarget", 0.92);
```

```
setValue("carbonTarget", 0.88);
```

```
setValue("ugfTarget", 0.86);
```

```
setValue("comfortTarget", 0.88);
```

```
setValue("renewableArea", 0.82);
```

```
}
```

```
if (m === "Low Carbon") {
```

```
setValue("carbonTarget", 0.94);
```

```
setValue("energyTarget", 0.84);
```

```
setValue("renewableArea", 0.82);
```

```
selects.viewMode.selected("Carbon Diagram");
```

```
}
```

```
if (m === "Comfort Priority") {
```

```
setValue("comfortTarget", 0.94);
```

```
setValue("shadeResponse", 0.88);
```

```
setValue("passiveFacade", 0.88);
```

```
setValue("balconyDepth", 2.4);
```

```
selects.viewMode.selected("Performance Diagram");
```

```
}
```

```
if (m === "Urban Greening") {
```

```
setValue("ugfTarget", 0.95);
```

```
    setValue("greenAmount", 0.92);

    setValue("treeCount", 22);

    setValue("waterReuse", 0.82);

    selects.viewMode.selected("Performance Diagram");

}

}
```

```
function setValue(key, val) {

    if (controls[key]) controls[key].value(val);

}
```

```
function randomiseMassing() {

    iterationIndex++;

    if (iterationIndex > maxIterations) iterationIndex = 1;
```

```
    seedValue = 1000 + iterationIndex * 137;

    applyIterationPreset(iterationIndex);
```

```
    savedIterationLog.push({

        iteration: iterationIndex,

        seed: seedValue,

        parameters: exportAllParameters(),

        performance: calculateOverallPerformance()

    });
```

```
    if (savedIterationLog.length > maxIterations) savedIterationLog.shift();

}
```

```

function applyIterationPreset(i) {

    setValue("density", 0.45 + hash01(i, 1, seedValue, 301) * 0.45);

    setValue("setback", 0.8 + hash01(i, 2, seedValue, 302) * 5.8);

    setValue("stagger", 0.2 + hash01(i, 3, seedValue, 303) * 1.8);

    setValue("courtyard", 0.18 + hash01(i, 4, seedValue, 304) * 0.58);

    setValue("voidStrength", 0.12 + hash01(i, 5, seedValue, 305) * 0.62);

    setValue("floors", floor(3 + hash01(i, 6, seedValue, 306) * 7));

    setValue("cols", floor(5 + hash01(i, 7, seedValue, 307) * 8));

    setValue("rows", floor(3 + hash01(i, 8, seedValue, 308) * 6));

    setValue("greenAmount", 0.35 + hash01(i, 9, seedValue, 309) * 0.57);

    setValue("screenAmount", 0.30 + hash01(i, 10, seedValue, 310) * 0.58);

    setValue("balconyDepth", 1.2 + hash01(i, 11, seedValue, 311) * 2.2);


    const modes = ["Balanced", "Maximum Performance", "Low Carbon", "Comfort Priority", "Urban Greening"];

    selects.optimisationMode.selected(modes[(i - 1) % modes.length]);

}


/* =====

EXPORTS: JSON, CSV, OBJ-LIKE, PNG

===== */


function exportAllParameters() {

    const params = {};

    for (const k in controls) params[k] = value(k);

    return params;

}

```

```

function ensureModuleDataForExport() {

  if (generatedModules.length > 0) return;


  generatedModules = [];


  const grid = getEffectiveGrid();

  const moduleW = mToP5(value("moduleW"));

  const moduleD = mToP5(value("moduleD"));

  const moduleH = mToP5(value("moduleH"));

  const gap = getSafeGap();

  const env = getBuildableEnvelope();

  const totalW = grid.cols * (moduleW + gap);

  const totalD = grid.rows * (moduleD + gap);

  const setback = mToP5(value("setback"));

  const stagger = min(mToP5(value("stagger")), gap * 0.85);


  for (let f = 0; f < value("floors"); f++) {

    for (let r = 0; r < grid.rows; r++) {

      for (let c = 0; c < grid.cols; c++) {

        if (!shouldPlaceModule(c, r, f, grid.cols, grid.rows, value("floors"))) continue;


        const pos = modulePosition(

          c, r, f, grid.cols, grid.rows,

          moduleW, moduleD, moduleH, gap,

          totalW, totalD, setback, stagger,

          env.x, env.z

        );

```

```
generatedModules.push(makeModuleData(c, r, f, pos, moduleW, moduleD, moduleH));  
  
    }  
  
    }  
  
    }  
  
}
```

```
function exportModuleJSON() {
```

```
    ensureModuleDataForExport();
```

```
    saveJSON({
```

```
        project: PROJECT_NAME,
```

```
        brand: BRAND_NAME,
```

```
        author: AUTHOR_NAME,
```

```
        scale: "1 p5 unit = 0.1 metre",
```

```
        iteration: iterationIndex,
```

```
        region: selects.region.value(),
```

```
        material: selects.materialSystem.value(),
```

```
        parameters: exportAllParameters(),
```

```
        performance: calculateOverallPerformance(),
```

```
        modules: generatedModules
```

```
    }, "urban_matrix_modular_housing_v2_1.json");
```

```
}
```

```
function exportCSVSchedules() {
```

```
    ensureModuleDataForExport();
```

```
    const lines = ["id,column,row,floor,x_p5,y_p5,z_p5,width_m,depth_m,height_m,type,material"];
```

```

generatedModules.forEach(m => {

  lines.push([

    m.id,

    m.column,

    m.row,

    m.floor,

    nf(m.x, 1, 2),

    nf(m.y, 1, 2),

    nf(m.z, 1, 2),

    m.widthM,

    m.depthM,

    m.heightM,

    m.type,

    m.materialSystem

  ].join(", "));

});

saveStrings(lines, "urban_matrix_module_schedule_v2_1.csv");

}

```

```

function exportOBJLike() {

  ensureModuleDataForExport();

  const lines = [

    "# Urban Matrix OBJ-like conceptual export",

    `# ${AUTHOR_NAME}`,

    "# Scale: 1 p5 unit = 0.1 metre"

  ];
}

```

```

generatedModules.forEach(m => {

    lines.push(

        `module ${m.id} x ${nf(m.x, 1, 2)} y ${nf(m.y, 1, 2)} z ${nf(m.z, 1, 2)} w ${nf(m.width, 1, 2)} h ${nf(m.height, 1, 2)} d
        ${nf(m.depth, 1, 2)}`

    );

});

```

```

saveStrings(lines, "urban_matrix_obj_like_v2_1.txt");

}

```

```

function saveCurrentViewPNG() {

    const oldClean = cleanMode;

    cleanMode = true;

    hudPanel.style("display", "none");

    uiRoot.style("opacity", "0");

    redraw();

    saveCanvas(`urban_matrix_current_view_option_${iterationIndex}`, "png");

    cleanMode = oldClean;

    hudPanel.style("display", cleanMode ? "none" : "block");

    uiRoot.style("opacity", cleanMode ? "0" : "1");

}

```

```

/* =====

```

2D DRAWING EXPORT ENGINE, PNG AND SVG

```

===== */

```

```

const sheetNames = [

    "Top Plan",

```

"Ground Floor Plan",

"Roof Plan",

"Site Plan",

"Masterplan",

"Front Elevation",

"Rear Elevation",

"Left Elevation",

"Right Elevation",

"Section A-A",

"Section B-B",

"Moodboard",

"Technical Sheet"

];

function exportSelectedDrawingPNG() {

exportDrawingPNG(selects.drawingType.value());

}

function exportSelectedDrawingSVG() {

exportDrawingSVG(selects.drawingType.value());

}

function exportAllDrawingPNGs() {

sheetNames.forEach(exportDrawingPNG);

}

function exportAllDrawingSVGs() {

sheetNames.forEach(exportDrawingSVG);

```
}
```

```
function exportDrawingPNG(type) {
```

```
  if (type === "Current 3D") {
```

```
    saveCurrentViewPNG();
```

```
    return;
```

```
  }
```

```
  ensureModuleDataForExport();
```

```
  const res = value("exportResolution");
```

```
  const g = createGraphics(1600 * res, 1000 * res);
```

```
  g.scale(res);
```

```
  drawSheet(g, type);
```

```
  save(g, `urban_matrix_${slug(type)}_v2_1.png`);
```

```
}
```

```
function exportDrawingSVG(type) {
```

```
  if (type === "Current 3D") type = "Top Plan";
```

```
  ensureModuleDataForExport();
```

```
  const svg = createSVGSheet(type);
```

```
  saveStrings(svg, `urban_matrix_${slug(type)}_v2_1.svg`);
```

```
}
```

```
function slug(s) {
```

```
  return s.toLowerCase().replace(/^[a-z0-9]+/g, "_").replace(/^\_|_$/g, "");
```

```
}
```

```
function drawSheet(g, type) {
```

```
  g.background(252);
```

```
  drawTitleBlock(g, type);
```

```
  if ([ "Top Plan", "Ground Floor Plan", "Roof Plan" ].includes(type)) drawPlanSheet(g, type);
```

```
  else if (type === "Site Plan") drawSitePlanSheet(g);
```

```
  else if (type === "Masterplan") drawMasterplanSheet(g);
```

```
  else if (type.includes("Elevation")) drawElevationSheet(g, type);
```

```
  else if (type.includes("Section")) drawSectionSheet(g, type);
```

```
  else if (type === "Moodboard") drawMoodboardSheet(g);
```

```
  else if (type === "Technical Sheet") drawTechnicalSheet(g);
```

```
  drawFooter(g, type);
```

```
}
```

```
function drawTitleBlock(g, type) {
```

```
  g.noFill();
```

```
  g.stroke(25);
```

```
  g.strokeWeight(1.4);
```

```
  g.rect(40, 40, 1520, 920);
```

```
  g.noStroke();
```

```
  g.fill(25);
```

```
  g.textSize(24);
```

```
  g.text(`${BRAND_NAME} | ${type}`, 60, 78);
```

```
g.textSize(12);
```

```
g.fill(70);
```

```
g.text(`Project: ${PROJECT_NAME}`, 60, 102);
```

```
g.text(`Designer: ${AUTHOR_NAME} | Scale note: 1 p5 unit = 0.1 metre | Option ${iterationIndex}/${maxIterations}`, 60, 120);
```

```
}
```

```
function drawFooter(g, type) {
```

```
g.noStroke();
```

```
g.fill(35);
```

```
g.textSize(10);
```

```
g.text(`${BRAND_NAME} | ${AUTHOR_NAME} | ${type} | Conceptual design export | Not for statutory approval`, 60, 940);
```

```
g.text(`Region: ${selects.region.value()} | Material: ${selects.materialSystem.value()} | LinkedIn: ${LINKEDIN_URL}`, 60, 958);
```

```
}
```

```
function drawingBounds(mode) {
```

```
ensureModuleDataForExport();
```

```
let xs = [];
```

```
let ys = [];
```

```
generatedModules.forEach(m => {
```

```
const w = mode === "side" ? m.depth : m.width;
```

```
const x = mode === "side" ? m.z : m.x;
```

```
xs.push(x - w / 2, x + w / 2);
```

```
ys.push(m.y - m.height / 2, m.y + m.height / 2);
```

```
});
```

```
return {  
  
    minX: min(xs),  
  
    maxX: max(xs),  
  
    minY: min(ys),  
  
    maxY: max(ys)  
  
};  
  
}
```

```
function fitTransform(bounds, box) {  
  
    const dw = bounds.maxX - bounds.minX;  
  
    const dh = bounds.maxY - bounds.minY;  
  
    const sc = min(box.w / max(1, dw), box.h / max(1, dh));  
  
    const ox = box.x + box.w / 2 - (bounds.minX + dw / 2) * sc;  
  
    const oy = box.y + box.h / 2 - (bounds.minY + dh / 2) * sc;  
  
    return { sc, ox, oy };  
  
}
```

```
function drawPlanSheet(g, type) {  
  
    const box = { x: 120, y: 170, w: 1000, h: 650 };  
  
    draw2DSiteBase(g, box, true);  
  
    draw2DModulesPlan(g, box, type);  
  
    drawPlanDimensions(g, box);  
  
    drawNorthArrow(g, 1410, 155);  
  
    drawScaleBar(g, 1240, 870, 100);  
  
    drawLegend(g, 1180, 210);  
  
}
```

```
function drawSitePlanSheet(g) {
```

```
const box = { x: 150, y: 185, w: 960, h: 620 };
```

```
draw2DSiteBase(g, box, true);
```

```
draw2DFootprint(g, box);
```

```
drawSideBoundaryLabels(g, box);
```

```
drawSetbackDimensions(g, box);
```

```
drawNorthArrow(g, 1410, 155);
```

```
drawScaleBar(g, 1240, 870, 100);
```

```
drawLegend(g, 1180, 210);
```

```
}
```

```
function drawMasterplanSheet(g) {
```

```
const box = { x: 100, y: 170, w: 1050, h: 650 };
```

```
draw2DSiteBase(g, box, true);
```

```
draw2DModulesPlan(g, box, "Roof Plan");
```

```
g.noStroke();
```

```
g.fill(205);
```

```
g.rect(1220, 190, 180, 130);
```

```
g.rect(1270, 650, 160, 130);
```

```
g.fill(50);
```

```
g.textSize(12);
```

```
g.text("Context Building", 1220, 340);
```

```
g.text("Neighbour Plot", 1270, 800);
```

```
drawNorthArrow(g, 1410, 155);
```

```
drawScaleBar(g, 1240, 870, 100);
```

```
drawLegend(g, 1180, 210);
```

```
}
```

```
function draw2DSiteBase(g, box, envelope) {
```

```
  const s = getSite();
```

```
  const sc = min(box.w / s.w, box.h / s.d);
```

```
  const sw = s.w * sc;
```

```
  const sd = s.d * sc;
```

```
  const cx = box.x + box.w / 2;
```

```
  const cy = box.y + box.h / 2;
```

```
  g.fill(238, 232, 220);
```

```
  g.stroke(35);
```

```
  g.strokeWeight(value("lineWeight"));
```

```
  g.rect(cx - sw / 2, cy - sd / 2, sw, sd);
```

```
  g.fill(...palette.road);
```

```
  g.noStroke();
```

```
  g.rect(cx - sw / 2, cy - sd / 2, s.roadW * sc, sd);
```

```
  g.fill(...palette.footpath);
```

```
  g.rect(cx - sw / 2 + s.roadW * sc, cy - sd / 2, s.footpathW * sc, sd);
```

```
  if (envelope) {
```

```
    const env = getBuildableEnvelope();
```

```
    g.noFill();
```

```
    g.stroke(...palette.envelope);
```

```
    g.strokeWeight(1.4);
```

```
    g.rect(cx + (env.x - env.w / 2) * sc, cy + (env.z - env.d / 2) * sc, env.w * sc, env.d * sc);
```

```
}
```

```
g.fill(35);
```

```
g.noStroke();
```

```
g.textSize(value("textSize2D"));
```

```
g.text(`Site: ${fmtM(s.wM)} × ${fmtM(s.dM)}`, cx - sw / 2, cy + sd / 2 + 24);
```

```
}
```

```
function draw2DFootprint(g, box) {
```

```
  const s = getSite();
```

```
  const sc = min(box.w / s.w, box.h / s.d);
```

```
  const cx = box.x + box.w / 2;
```

```
  const cy = box.y + box.h / 2;
```

```
  const b = planModuleBounds();
```

```
  g.noFill();
```

```
  g.stroke(30);
```

```
  g.strokeWeight(2);
```

```
  g.rect(cx + b.minX * sc, cy + b.minZ * sc, (b.maxX - b.minX) * sc, (b.maxZ - b.minZ) * sc);
```

```
  g.noStroke();
```

```
  g.fill(30);
```

```
  g.textSize(11);
```

```
  g.text("Building Footprint", cx + b.minX * sc, cy + b.minZ * sc - 8);
```

```
}
```

```
function planModuleBounds() {
```

```
  ensureModuleDataForExport();
```

```
const xs = [];
```

```
const zs = [];
```

```
generatedModules.forEach(m => {
```

```
  xs.push(m.x - m.width / 2, m.x + m.width / 2);
```

```
  zs.push(m.z - m.depth / 2, m.z + m.depth / 2);
```

```
});
```

```
return {
```

```
  minX: min(xs),
```

```
  maxX: max(xs),
```

```
  minZ: min(zs),
```

```
  maxZ: max(zs)
```

```
};
```

```
}
```

```
function draw2DModulesPlan(g, box, type) {
```

```
  const s = getSite();
```

```
  const sc = min(box.w / s.w, box.h / s.d);
```

```
  const cx = box.x + box.w / 2;
```

```
  const cy = box.y + box.h / 2;
```

```
  const targetFloor = type === "Ground Floor Plan" ? 0 : value("floors") - 1;
```

```
  generatedModules.forEach(m => {
```

```
    if (type !== "Top Plan" && m.floor !== targetFloor) return;
```

```
    if (type === "Top Plan" && m.floor < value("floors") - 3 && hash01(m.column, m.row, m.floor, 8) > 0.35) return;
```

```
const x = cx + (m.x - m.width / 2) * sc;
```

```
const z = cy + (m.z - m.depth / 2) * sc;
```

```
const w = m.width * sc;
```

```
const d = m.depth * sc;
```

```
g.fill(216, 184, 145);
```

```
g.stroke(55);
```

```
g.strokeWeight(1);
```

```
g.rect(x, z, w, d);
```

```
if (type !== "Ground Floor Plan") {
```

```
  g.fill(...palette.grass);
```

```
  g.noStroke();
```

```
  g.rect(x + w * 0.28, z + d * 0.28, w * 0.44, d * 0.44);
```

```
} else {
```

```
  g.fill(120, 154, 164);
```

```
  g.noStroke();
```

```
  g.rect(x + w * 0.35, z + d * 0.35, w * 0.30, d * 0.30);
```

```
}
```

```
});
```

```
}
```

```
function drawPlanDimensions(g, box) {
```

```
  if (!checked("dimensions")) return;
```

```
  const s = getSite();
```

```
  const b = planModuleBounds();
```

```
  const sc = min(box.w / s.w, box.h / s.d);
```

```
const cx = box.x + box.w / 2;
```

```
const cy = box.y + box.h / 2;
```

```
dimensionLine(g, cx + b.minX * sc, cy + b.maxZ * sc + 30, cx + b.maxX * sc, cy + b.maxZ * sc + 30, fmtM(p5ToM(b.maxX - b.minX)));
```

```
dimensionLine(g, cx + b.maxX * sc + 35, cy + b.minZ * sc, cx + b.maxX * sc + 35, cy + b.maxZ * sc, fmtM(p5ToM(b.maxZ - b.minZ)));
```

```
}
```

```
function drawSetbackDimensions(g, box) {
```

```
  if (!checked("dimensions")) return;
```

```
  const s = getSite();
```

```
  const sc = min(box.w / s.w, box.h / s.d);
```

```
  const cx = box.x + box.w / 2;
```

```
  const cy = box.y + box.h / 2;
```

```
  const sw = s.w * sc;
```

```
  const sd = s.d * sc;
```

```
  g.fill(35);
```

```
  g.textSize(11);
```

```
  g.text(`Front setback: ${fmtM(s.frontM)}`, cx - sw / 2 + 260, cy - sd / 2 - 18);
```

```
  g.text(`Rear setback: ${fmtM(s.rearM)}`, cx - sw / 2 + 260, cy + sd / 2 + 42);
```

```
  g.text(`Side setbacks: L ${fmtM(s.leftM)} / R ${fmtM(s.rightM)}`, cx - sw / 2 + 260, cy + sd / 2 + 60);
```

```
}
```

```
function drawSideBoundaryLabels(g, box) {
```

```
  const s = getSite();
```

```
  const sc = min(box.w / s.w, box.h / s.d);
```

```
const cx = box.x + box.w / 2;
```

```
const cy = box.y + box.h / 2;
```

```
const sw = s.w * sc;
```

```
const sd = s.d * sc;
```

```
g.fill(...palette.boundary);
```

```
g.noStroke();
```

```
g.textSize(12);
```

```
g.text("SIDE A / ROAD FRONTAGE", cx - sw / 2 + 20, cy - sd / 2 - 10);
```

```
g.text("SIDE C / REAR BOUNDARY", cx - sw / 2 + 20, cy + sd / 2 + 20);
```

```
g.text("SIDE D", cx - sw / 2 - 45, cy);
```

```
g.text("SIDE B", cx + sw / 2 + 10, cy);
```

```
}
```

```
function drawElevationSheet(g, type) {
```

```
  const side = type.split(" ")[0].toLowerCase();
```

```
  const mode = (side === "left" || side === "right") ? "side" : "front";
```

```
  const b = drawingBounds(mode);
```

```
  const box = { x: 120, y: 200, w: 1120, h: 610 };
```

```
  const t = fitTransform(b, box);
```

```
  drawLevelLines(g, box, t);
```

```
  generatedModules.forEach(m => {
```

```
    const faceW = mode === "side" ? m.depth : m.width;
```

```
    const xAxis = mode === "side" ? m.z : m.x;
```

```
    const x = t.ox + (xAxis - faceW / 2) * t.sc;
```

```
    const y = t.oy + (m.y - m.height / 2) * t.sc;
```

```
const w = faceW * t.sc;
```

```
const h = m.height * t.sc;
```

```
g.fill(216, 184, 145);
```

```
g.stroke(55);
```

```
g.rect(x, y, w, h);
```

```
g.fill(...palette.glass);
```

```
g.rect(x + w * 0.36, y + h * 0.32, w * 0.28, h * 0.22);
```

```
});
```

```
drawElevationDimensions(g, box, t);
```

```
drawScaleBar(g, 1240, 870, 100);
```

```
g.fill(40);
```

```
g.noStroke();
```

```
g.textSize(13);
```

```
g.text(`${type.toUpperCase()} | floor levels, module grid, openings and overall height`, 60, 155);
```

```
}
```

```
function drawLevelLines(g, box, t) {
```

```
const h = mToP5(value("moduleH"));
```

```
g.stroke(160);
```

```
g.strokeWeight(0.8);
```

```
g.fill(55);
```

```
g.textSize(10);
```

```
for (let f = 0; f <= value("floors"); f++) {
```

```

const y = t.oy + (groundY - f * h) * t.sc;

g.line(box.x, y, box.x + box.w, y);

g.noStroke();

g.text(f === 0 ? "Ground" : (f === value("floors") ? "Roof" : `L${f}`), box.x + box.w + 12, y + 4);

g.stroke(160);

}

}

```

```

function drawElevationDimensions(g, box, t) {

```

```

  if (!checked("dimensions")) return;

```

```

  const overallH = value("moduleH") * value("floors");

```

```

  dimensionLine(

```

```

    g,

```

```

    box.x + box.w + 85,

```

```

    t.oy + groundY * t.sc,

```

```

    box.x + box.w + 85,

```

```

    t.oy + (groundY - mToP5(overallH)) * t.sc,

```

```

    `Overall height ${fmtM(overallH)}`

```

```

  );

```

```

  g.fill(35);

```

```

  g.noStroke();

```

```

  g.textSize(11);

```

```

  g.text(`Floor-to-floor: ${fmtM(value("moduleH"))}`, box.x, box.y + box.h + 38);

```

```

}

```

```

function drawSectionSheet(g, type) {

```

```
const mode = type === "Section A-A" ? "front" : "side";
```

```
const b = drawingBounds(mode === "side" ? "side" : "front");
```

```
const box = { x: 120, y: 200, w: 1120, h: 610 };
```

```
const t = fitTransform(b, box);
```

```
drawLevelLines(g, box, t);
```

```
generatedModules.forEach(m => {
```

```
  const include = type === "Section A-A"
```

```
    ? abs(m.row - floor(getEffectiveGrid().rows / 2)) <= 1
```

```
    : abs(m.column - floor(getEffectiveGrid().cols / 2)) <= 1;
```

```
  if (!include) return;
```

```
  const faceW = mode === "side" ? m.depth : m.width;
```

```
  const xAxis = mode === "side" ? m.z : m.x;
```

```
  g.fill(70);
```

```
  g.stroke(35);
```

```
  g.rect(
```

```
    t.ox + (xAxis - faceW / 2) * t.sc,
```

```
    t.oy + (m.y - m.height / 2) * t.sc,
```

```
    faceW * t.sc,
```

```
    m.height * t.sc
```

```
  );
```

```
});
```

```
drawElevationDimensions(g, box, t);
```

```
drawScaleBar(g, 1240, 870, 100);
```

```
g.fill(40);
```

```
g.noStroke();
```

```
g.textSize(13);
```

```
g.text(`${type.toUpperCase()} | conceptual cut line, modules, floor levels and overall height`, 60, 155);
```

```
}
```

```
function drawMoodboardSheet(g) {
```

```
g.fill(40);
```

```
g.textSize(13);
```

```
g.text("MOODBOARD | Material palette, climate response and sustainability logic", 60, 155);
```

```
const names = Object.keys(materialProfiles);
```

```
names.forEach((n, i) => {
```

```
  const m = materialProfiles[n];
```

```
  const x = 90 + i * 360;
```

```
  const y = 200;
```

```
g.fill(...m.colour);
```

```
g.noStroke();
```

```
g.rect(x, y, 260, 150);
```

```
g.fill(30);
```

```
g.textSize(13);
```

```
g.text(n, x, y + 178);
```

```
g.textSize(10);
```

```
g.text(m.structure, x, y + 198, 270, 70);
```

```
});
```

```
const swatches = [
```

```
  ["Concrete", palette.concrete],
```

```
  ["Clay", palette.clay],
```

```
  ["Timber", palette.timber],
```

```
  ["Green Roof", palette.grass],
```

```
  ["Glass", palette.glass],
```

```
  ["PV", palette.pv],
```

```
  ["Water", palette.water]
```

```
];
```

```
swatches.forEach((s, i) => {
```

```
  const x = 90 + i * 200;
```

```
  const y = 560;
```

```
    g.fill(...s[1]);
```

```
    g.rect(x, y, 140, 85);
```

```
    g.fill(30);
```

```
    g.textSize(11);
```

```
    g.text(s[0], x, y + 110);
```

```
  });
```

```
}
```

```
function drawTechnicalSheet(g) {
```

```
  g.fill(40);
```

```
g.textSize(13);
```

```
g.text("TECHNICAL CONCEPT SHEET | conceptual details, not construction details", 60, 155);
```

```
typicalModuleDetail(g, 90, 210);
```

```
greenRoofDetail(g, 470, 210);
```

```
facadeScreenDetail(g, 850, 210);
```

```
balconyDetail(g, 1230, 210);
```

```
pvWaterDetail(g, 90, 600);
```

```
}
```

```
function typicalModuleDetail(g, x, y) {
```

```
  g.fill(230, 210, 180);
```

```
  g.stroke(40);
```

```
  g.rect(x, y, 260, 150);
```

```
  g.fill(...palette.glass);
```

```
  g.rect(x + 90, y + 48, 80, 55);
```

```
  g.fill(30);
```

```
  g.noStroke();
```

```
  g.textSize(12);
```

```
  g.text("Typical module", x, y - 12);
```

```
  g.text(`W ${fmtM(value("moduleW"))} / D ${fmtM(value("moduleD"))} / H ${fmtM(value("moduleH"))}`, x, y + 180);
```

```
}
```

```
function greenRoofDetail(g, x, y) {
```

```
  g.fill(210, 190, 160);
```

```
  g.rect(x, y + 120, 260, 28);
```

```
  g.fill(90, 90, 80);
```

```
  g.rect(x + 20, y + 98, 220, 22);
```

```
g.fill(...palette.grass);
```

```
g.rect(x + 20, y + 60, 220, 38);
```

```
g.fill(30);
```

```
g.textSize(12);
```

```
g.text("Green roof build-up", x, y - 12);
```

```
g.text("Vegetation / substrate / drainage / waterproofing / slab", x, y + 180, 300, 40);
```

```
}
```

```
function facadeScreenDetail(g, x, y) {
```

```
g.fill(230, 210, 180);
```

```
g.rect(x, y, 260, 150);
```

```
g.fill(...palette.timber);
```

```
for (let i = 0; i < 9; i++) {
```

```
g.rect(x + 45 + i * 20, y + 28, 6, 95);
```

```
}
```

```
g.fill(30);
```

```
g.textSize(12);
```

```
g.text("Façade fins / jaali logic", x, y - 12);
```

```
g.text("Shading, privacy, identity", x, y + 180);
```

```
}
```

```
function balconyDetail(g, x, y) {
```

```
g.fill(230, 210, 180);
```

```
g.rect(x, y + 85, 240, 25);
```

```
g.fill(...palette.rail);
```

```
g.rect(x + 20, y + 25, 200, 8);
```

```
g.rect(x + 20, y + 25, 8, 65);

g.rect(x + 212, y + 25, 8, 65);

g.fill(30);

g.textSize(12);

g.text("Balcony concept", x, y - 12);

g.text(`Projection ${fmtM(value("balconyDepth"))}`, x, y + 180);

}
```

```
function pvWaterDetail(g, x, y) {
```

```
g.fill(...palette.pv);
```

```
for (let i = 0; i < 6; i++) {
```

```
g.rect(x + i * 55, y, 45, 28);
```

```
}
```

```
g.fill(...palette.water);
```

```
g.ellipse(x + 90, y + 120, 70, 70);
```

```
g.fill(30);
```

```
g.textSize(12);
```

```
g.text("PV array + rainwater reuse", x, y - 12);
```

```
g.text("PV roof zone, tank, reuse and maintenance access required", x, y + 180);
```

```
}
```

```
function dimensionLine(g, x1, y1, x2, y2, label) {
```

```
g.stroke(30);
```

```
g.strokeWeight(1.1);
```

```
g.line(x1, y1, x2, y2);
```

```
g.noStroke();
```

```
g.fill(30);
```

```
g.textSize(11);
```

```
g.text(label, (x1 + x2) / 2 + 8, (y1 + y2) / 2 - 6);
```

```
}
```

```
function drawNorthArrow(g, x, y) {
```

```
g.fill(25);
```

```
g.noStroke();
```

```
g.textSize(12);
```

```
g.text("N", x - 4, y - 30);
```

```
g.triangle(x, y - 20, x - 10, y + 25, x + 10, y + 25);
```

```
}
```

```
function drawScaleBar(g, x, y, len) {
```

```
g.stroke(25);
```

```
g.strokeWeight(4);
```

```
g.line(x, y, x + len, y);
```

```
g.noStroke();
```

```
g.fill(25);
```

```
g.textSize(11);
```

```
g.text("10 m", x + len + 8, y + 4);
```

```
}
```

```
function drawLegend(g, x, y) {
```

```
const items = [
```

```
  ["Site boundary", [238, 232, 220]],
```

```
  ["Buildable envelope", palette.envelope],
```

```
  ["Road", palette.road],
```

```
["Footpath", palette.footpath],
```

```
["Green roof", palette.grass],
```

```
["PV panels", palette.pv],
```

```
["Context", [205, 205, 198]]
```

```
];
```

```
g.fill(30);
```

```
g.textSize(13);
```

```
g.text("Legend", x, y);
```

```
items.forEach((it, i) => {
```

```
  g.fill(...it[1]);
```

```
  g.rect(x, y + 20 + i * 24, 18, 18);
```

```
  g.fill(35);
```

```
  g.textSize(11);
```

```
  g.text(it[0], x + 28, y + 34 + i * 24);
```

```
});
```

```
}
```

```
/* =====
```

```
SVG EXPORT
```

```
===== */
```

```
function createSVGSheet(type) {
```

```
  const svg = [];
```

```
  svg.push(<svg xmlns="http://www.w3.org/2000/svg" width="1600" height="1000" viewBox="0 0 1600 1000">`);
```

```
  svg.push(<rect width="1600" height="1000" fill="#fcfcfc"/>`);
```

```

svg.push(`<rect x="40" y="40" width="1520" height="920" fill="none" stroke="#111" stroke-width="1.4"/>`);

svg.push(`<text x="60" y="78" font-family="Arial" font-size="24" fill="#222">${escapeXML(BRAND_NAME)} |
${escapeXML(type)}</text>`);

svg.push(`<text x="60" y="104" font-family="Arial" font-size="12" fill="#555">Project:
${escapeXML(PROJECT_NAME)}</text>`);

svg.push(`<text x="60" y="122" font-family="Arial" font-size="12" fill="#555">Designer:
${escapeXML(AUTHOR_NAME)} | Scale note: 1 p5 unit = 0.1 metre | Option ${iterationIndex}/${maxIterations}</text>`);


if ([ "Top Plan", "Ground Floor Plan", "Roof Plan", "Site Plan", "Masterplan" ].includes(type)) svgPlan(svg, type);

else if (type.includes("Elevation")) svgElevation(svg, type);

else if (type.includes("Section")) svgSection(svg, type);

else svg.push(`<text x="90" y="180" font-family="Arial" font-size="14">SVG vector export available for technical drafting.
PNG sheet contains full moodboard/detail graphics.</text>`);


svg.push(`<text x="60" y="940" font-family="Arial" font-size="10" fill="#222">${escapeXML(BRAND_NAME)} |
${escapeXML(AUTHOR_NAME)} | ${escapeXML(type)} | Conceptual design export | Not for statutory approval</text>`);

svg.push(`</svg>`);


return svg;

}


function svgPlan(svg, type) {

const s = getSite();

const box = { x: 150, y: 185, w: 960, h: 620 };

const sc = Math.min(box.w / s.w, box.h / s.d);

const cx = box.x + box.w / 2;

const cy = box.y + box.h / 2;

const sw = s.w * sc;

const sd = s.d * sc;

svg.push(rect(cx - sw / 2, cy - sd / 2, sw, sd, "#eee8dc", "#222", 1));

```

```
svg.push(rect(cx - sw / 2, cy - sd / 2, s.roadW * sc, sd, "#555", "none", 0));
```

```
svg.push(rect(cx - sw / 2 + s.roadW * sc, cy - sd / 2, s.footpathW * sc, sd, "#e6d8c2", "none", 0));
```

```
const env = getBuildableEnvelope();
```

```
svg.push(rect(cx + (env.x - env.w / 2) * sc, cy + (env.z - env.d / 2) * sc, env.w * sc, env.d * sc, "none", "#469462", 2));
```

```
if (type !== "Site Plan") {
```

```
  generatedModules.forEach(m => {
```

```
    if (type !== "Top Plan" && m.floor !== (type === "Ground Floor Plan" ? 0 : value("floors") - 1)) return;
```

```
    const x = cx + (m.x - m.width / 2) * sc;
```

```
    const z = cy + (m.z - m.depth / 2) * sc;
```

```
    svg.push(rect(x, z, m.width * sc, m.depth * sc, "#d8b891", "#333", 1));
```

```
    if (type !== "Ground Floor Plan") {
```

```
      svg.push(rect(x + m.width * sc * 0.28, z + m.depth * sc * 0.28, m.width * sc * 0.44, m.depth * sc * 0.44, "#7eab7a", "none", 0));
```

```
    }
```

```
  });
```

```
}
```

```
svg.push(`<text x="${cx - sw / 2}" y="${cy + sd / 2 + 28}" font-family="Arial" font-size="12">Site: ${fmtM(s.wM)} ×  
${fmtM(s.dM)}</text>`);
```

```
}
```

```
function svgElevation(svg, type) {
```

```
  const mode = (type.startsWith("Left") || type.startsWith("Right")) ? "side" : "front";
```

```
  const b = drawingBounds(mode);
```

```
const box = { x: 120, y: 200, w: 1120, h: 610 };
```

```
const t = fitTransform(b, box);
```

```
generatedModules.forEach(m => {
```

```
  const faceW = mode === "side" ? m.depth : m.width;
```

```
  const xAxis = mode === "side" ? m.z : m.x;
```

```
    svg.push(rect(
```

```
      t.ox + (xAxis - faceW / 2) * t.sc,
```

```
      t.oy + (m.y - m.height / 2) * t.sc,
```

```
      faceW * t.sc,
```

```
      m.height * t.sc,
```

```
      "#d8b891",
```

```
      "#333",
```

```
      1
```

```
    ));
```

```
  });
```

```
    svg.push(`<text x="60" y="155" font-family="Arial" font-size="13">${escapeXML(type.toUpperCase())} | levels, module  
grid and overall height</text>`);
```

```
  }
```

```
function svgSection(svg, type) {
```

```
  svgElevation(svg, type);
```

```
}
```

```
function rect(x, y, w, h, fillValue, strokeValue, sw) {
```

```
  return `<rect x="${nf(x, 1, 2)}" y="${nf(y, 1, 2)}" width="${nf(w, 1, 2)}" height="${nf(h, 1, 2)}" fill="${fillValue}"  
stroke="${strokeValue}" stroke-width="${sw}" />`;
```

```
}
```

```
function escapeXML(s) {
```

```
  return String(s).replaceAll("&", "&amp;").replaceAll("<", "&lt;").replaceAll(">", "&gt;");
```

```
}
```

```
/* =====
```

```
HUD, LIGHTS, INPUT AND UTILITIES
```

```
===== */
```

```
function updateHUD() {
```

```
  if (!hudPanel || cleanMode) return;
```

```
  const grid = getEffectiveGrid();
```

```
  const perf = calculateOverallPerformance();
```

```
  const s = getSite();
```

```
  hudPanel.html(
```

```
    `<b>${BRAND_NAME} | ${AUTHOR_NAME}</b><br>` +
```

```
    `${PROJECT_NAME}<br>` +
```

```
    `Scale: <b>1 p5 unit = 0.1 m</b><br>` +
```

```
    `Option: ${iterationIndex}/${maxIterations}<br>` +
```

```
    `Site: ${fmtM(s.wM)} × ${fmtM(s.dM)} | Grid: ${grid.cols} × ${grid.rows} × ${value("floors")}<br>` +
```

```
    `Modules: ${generatedModules.length} | Score: ${nf(perf.total * 100, 1, 0)}%<br>` +
```

```
    `Drawing view: ${selects.drawingType.value()}<br>` +
```

```
    `${boundaryWarning ? '<span style="color:#ffb199">Boundary warning: auto-fit limiting rows/columns.</span>' :  
'Boundary and spacing active.'}<br>` +
```

```
    `<span style="font-size:10px">H hide controls | C clean mode | R iterate | SVG exports are vector drawings</span>`
```

```
  );
```

```
}
```

```
function setupLights() {
```

```
  if (selects.viewMode.value() === "Night") {
```

```
    ambientLight(60, 66, 75);
```

```
    pointLight(255, 220, 170, -350, -260, 260);
```

```
  }
  return;
```

```
}
```

```
  ambientLight(154, 150, 142);
```

```
  const az = radians(value("sunAzimuth"));
```

```
  const alt = radians(value("sunAltitude"));
```

```
  directionalLight(255, 232, 190, -cos(az) * cos(alt), -sin(alt), -sin(az) * cos(alt));
```

```
  directionalLight(160, 180, 190, 0.6, -0.35, 0.7);
```

```
}
```

```
function resetCamera() {
```

```
  camYaw = -0.72;
```

```
  camPitch = -0.48;
```

```
  camDist = 1080;
```

```
  camTarget = { x: 90, y: -150, z: 20 };
```

```
  selects.drawingType.selected("Current 3D");
```

```
}
```

```
function keyPressed() {
```

```
  if (key === "h" || key === "H") toggleUIVisibility();
```

```
if (key === "c" || key === "C") toggleCleanMode();
```

```
if (key === "r" || key === "R") randomiseMassing();
```

```
if (key === "s" || key === "S") exportModuleJSON();
```

```
if (key === "o" || key === "O") exportOBJLike();
```

```
if (key === "p" || key === "P") saveCurrentViewPNG();
```

```
if (key === "+" || key === "=") camDist = constrain(camDist - 70, 260, 3200);
```

```
if (key === "-" || key === "_") camDist = constrain(camDist + 70, 260, 3200);
```

```
}
```

```
function hash01(a, b, c, d) {
```

```
  const x = Math.sin(a * 127.1 + b * 311.7 + c * 74.7 + d * 43.3 + seedValue * 0.13) * 43758.5453123;
```

```
  return x - Math.floor(x);
```

```
}
```

```
function windowResized() {
```

```
  resizeCanvas(windowWidth, windowHeight);
```

```
}
```